

Algorithms Portfolio

Curated algorithmic solutions with correctness and complexity analysis

Ryan McMillan

MCompSci (cand.) – University of New England

C++ primary • Python secondary

January 27, 2026

License

This work is licensed under the MIT License. The canonical source and latest version are available at: <https://github.com/rmcmillan34/algorithms-portfolio>.

Glossary

Amortized analysis

An analysis technique that considers the average cost of operations over a sequence of operations, even if individual operations may be expensive.

Invariant

A property that remains true before and after each iteration of an algorithm or loop.

In-place algorithm

An algorithm that transforms its input using only a constant amount of additional memory.

Loop invariant

An invariant associated with a loop, used to prove correctness via initialization, maintenance, and termination.

Time complexity

An asymptotic measure of how an algorithm's running time grows as a function of input size.

Contents

Glossary	2
1 Introduction	4
1.1 How to Use This Book	4
1.2 Motivation	4
1.3 Acknowledgements	4
2 Arrays & Sequences	6
2.1 Core Ideas	6
2.2 Common Pitfalls	6
2.3 Primer - Arrays & Sequences	6
2.4 Easy	8
2.4.1 Plus One	8
3 Strings & Parsing	10
3.1 Core Ideas	10
3.2 Common Pitfalls	10
3.3 Easy	10
3.3.1 Valid Palindrome	10

1 Introduction

This document is a curated portfolio of algorithmic solutions. Each selected problem includes:

- clear problem restatements,
- key invariants / observations,
- pseudocode,
- correctness sketches,
- complexity analysis,
- C++ (primary) and Python (secondary) implementations.

The focus is on clarity, correctness, and reasoning.

1.1 How to Use This Book

This book is not a tutorial on data structures and algorithms, nor does it attempt to replace formal coursework or foundational texts. Instead, it serves as a refresher and practical reference, organised around LeetCode-style problems that commonly appear in technical interviews and applied programming contexts.

The emphasis is on recognising patterns, recalling standard techniques, and reviewing concise, correct implementations. Explanations focus on the idea being exercised and the invariants maintained, rather than on full theoretical development.

Readers are assumed to already be familiar with fundamental data structures and algorithmic concepts. The book is intended to be used for revision, reinforcement, and interview preparation, particularly when revisiting topics after time away from formal study.

Problems include an optional **Author's Thinking** block, the authors plain language thinking and problem framing, which is excluded from upstream exemplar builds. Use this book as an example, to understand how to formally reason with, and structure algorithm and data structures problems.

1.2 Motivation

This portfolio was created as a structured, self-directed study of fundamental algorithms and data structures. While my formal coursework emphasizes applied computer science and systems-level engineering, it does not include a dedicated course focused on algorithmic problem-solving in the style commonly expected in technical interviews and quantitative software roles.

To address this, I undertook a systematic review of core algorithmic techniques, organizing problems by data structure and algorithmic paradigm, and documenting each solution with an explicit algorithm description, correctness argument, and complexity analysis. The goal is not only to arrive at correct implementations, but to practice communicating algorithmic reasoning clearly and rigorously.

This document serves both as a personal reference and as evidence of deliberate practice in algorithmic reasoning beyond formal coursework.

Many of the problems included here are sourced from commonly used interview problem sets, and were selected to emphasize invariant-based reasoning, edge case analysis, and asymptotic performance trade-offs.

Informal reasoning notes are included selectively to document thought process and learning progression; these sections are omitted in derived exemplar or presentation-focused versions of this material.

1.3 Acknowledgements

This work was informed by a number of open educational resources that emphasize rigorous algorithmic reasoning and clear exposition. In particular, the structure and approach to cor-

rectness arguments and asymptotic analysis were influenced by *MIT OpenCourseWare 6.006: Introduction to Algorithms*.

Additional reference was made to publicly available problem discussions, editorials, and textbooks where appropriate. All implementations, explanations, and written analyses in this document are my own, and any external influences are acknowledged at a conceptual level rather than reproduced directly.

2 Arrays & Sequences

Arrays are the foundational data structure in most algorithmic problems. They provide constant-time indexed access and strong cache locality, but require careful reasoning about indices, bounds, and in-place mutation.

This chapter focuses on recognizing array-based problem patterns and maintaining clear invariants over contiguous memory.

2.1 Core Ideas

- Index-as-state reasoning
- In-place mutation vs auxiliary storage
- Prefix and suffix accumulation
- Simulation of elementary operations (e.g. addition, carry propagation)

2.2 Common Pitfalls

- Off-by-one errors at boundaries
- Unintended data overwrites during in-place updates
- Assuming sorting or uniqueness without verifying constraints
- Over-allocating space when a single pass suffices

2.3 Primer - Arrays & Sequences

What It Is

sed.). An array is a *contiguous* block of memory storing elements of the same type, supporting $O(1)$ access by index. A dynamic array (e.g. C++ `std::vector`, Python `list`) grows by allocating a larger block and copying elements when capacity is exceeded, providing *amortized* $O(1)$ append. A *sequence* generalises the idea to any indexable or iterable ordered collection; here we emphasise contiguous arrays.

Recognition Patterns

- Index math across contiguous elements (prefix/suffix, window, ranges).
- Constraints emphasising in-place mutation, $O(1)$ extra space, single pass
- Repeated accumulation or difference of segments (prefix sums/scans).
- Local adjacency decisions: next/previous comparison, runs, peaks/valleys

Core Operations

```

1 access(A, i):
2     return A[i] // O(1)
3
4 traverse(A):
5     for i from 0 to n-1:
6         visit(A[i])
7
8 insert_at(A, i, x):
9     grow A by 1
10    for j from n-1 downto i:
11        A[j+1] = A[j]
12    A[i] = x
13
14 erase_at(A, i):
15    for j from i to n-2:
16        A[j] = A[j+1]
17    shrink A by 1
18
19 prefix_sums(A):
20    P[0] = 0
21    for i from 0 to n-1:
22        P[i+1] = P[i] + A[i]
23    return P
24
25 two_pointers_sorted(A, target):
26    i = 0; j = n - 1
27    while i < j:
28        s = A[i] + A[j]
29        if s == target: return (i, j)
30        if s < target: i = i + 1 else: j = j - 1
31    return not found
32
33 sliding_window(A):
34    l = 0
35    for r from 0 to n-1:
36        add(A[r])
37        while window_invalid():
38            remove(A[l])
39            l = l + 1
40    update_answer()

```

Listing 1: Arrays: Core Operations (pseudocode)

Invariants & Correctness

Array scans typically maintain index-based invariants: (i) after i steps, positions $[0..i)$ have been processed, (ii) two-pointer: indices satisfy $0 \leq i \leq j < n$ and a monotone move (left increases or right decreases) prevents rework, (iii) sliding window: the maintained sub-array $[l, r)$ satisfies a property (e.g. distinct elements, $\text{sum} \leq K$); expansion and contraction preserve the property.

Complexity

- Access: time $O(1)$; Traversal: $O(n)$
- Insert/delete at end (dynamic array): amortised $O(1)$, at index $O(n)$
- Prefix sums / scan: $O(n)$ time, $O(n)$ extra space (or $O(1)$ in place if possible)
- Two-pointers / sliding window: typically $O(n)$ if each pointer advances monotonically.

Common Pitfalls

- Off-by-one at boundaries; confusing inclusive/exclusive ranges.
 - Forgetting to update all window state on both expand and shrink
 - Overflow in sums/products; use wider types for accumulation.
 - Accidental $O(n^2)$ from repeated slicing/copying in inner loops.
 - Assuming sortedness/uniqueness without verifying constraints.
-)

2.4 Easy

The following problems introduce fundamental array manipulation techniques. The emphasis is on establishing and maintaining simple invariants while iterating through the array.

2.4.1 Plus One

Difficulty: Easy

Tags: Arrays, Simulation

ID: LC-066

Problem. Given a non-empty array of digits representing a non-negative integer, increment the integer by one and return the resulting array of digits.

The most significant digit is stored at the beginning of the array, and each digit is in the range $[0, 9]$.

Key idea. Incrementing the number requires simulating elementary addition from right to left. Carry propagation only affects trailing digits equal to nine; once a digit strictly less than nine is incremented, the process terminates.

Author's Thinking.

My initial instinct was to convert the digit array into an integer, add one, and convert it back. This approach fails due to potential overflow and violates the problem constraints.

What simplified the reasoning was treating the operation as grade-school addition and maintaining the invariant that all digits to the right of the current index are already correct after carry handling.

Algorithm.

```
1 for i from n - 1 downto 0:
2     if digits[i] < 9:
3         digits[i] = digits[i] + 1
4         return digits
5     digits[i] = 0
6
7 prepend 1 to digits
8 return digits
```

Listing 2: Pseudocode

Correctness sketch. We iterate from the least significant digit toward the most significant digit. At each iteration, the invariant maintained is that all digits to the right of the current index represent the correct result after increment and carry propagation.

If a digit less than nine is encountered, incrementing it produces the correct result and no further carry is required. If all digits are nine, each is set to zero and a leading one is prepended, yielding the correct incremented number. Thus, the algorithm always returns the correct result.

Complexity. The algorithm runs in $O(n)$ time, where n is the number of digits. It uses $O(1)$ additional space beyond the output array.

C++ Implementation

```
1  /*
2  * LC-066 - Plus One
3  *
4  */
5
6  #include <iostream>
7
8  class Solution {
9  public:
10     vector<int> plusOne(vector<int>& digits) {
11         for(int i = digits.size(); i-- > 0 ; ) {
12             if (digits[i] != 9) {
13                 digits[i]++;
14                 return digits;
15             }
16             else {
17                 digits[i] = 0;
18             }
19         }
20
21         vector<int> newDigits(digits.size() + 1);
22         newDigits[0] = 1;
23         for (int j = 1; j < newDigits.size(); j++) {
24             newDigits[j] = digits[j-1];
25         }
26
27         return newDigits;
28     }
29 };
```

Listing 3: C++ Implementation

Python Implementation

```
1  # LC-066 Plus One
2
3  from typing import List
4
5  def plusOne(digits: List[int]) -> List[int]:
6      for i in range(len(digits) - 1, -1, -1):
7          if digits[i] < 9:
8              digits[i] += 1
9              return digits
10             digits[i] = 0
11
12     return [1] + digits
```

Listing 4: Python Implementation

3 Strings & Parsing

Strings are a fundamental data structure used to represent text and sequences of characters. They often require specialized algorithms for efficient manipulation, searching, and pattern matching. This chapter focuses on recognizing string-based problem patterns and maintaining clear invariants over character sequences.

3.1 Core Ideas

- Two-pointer techniques for substring problems
- Sliding window approaches for dynamic substring analysis
- Character classification and normalisation
- Maintaining loop invariants over indices
- Incremental Parsing
- In-place vs Auxiliary Storage

3.2 Common Pitfalls

- Off-by-one errors at boundaries
- Incorrect handling of empty strings or single-character strings
- Failing to advance pointers correctly when skipping characters
- Unnecessary string copying leading to increased time/space complexity
- Assumptions about character sets (e.g., ASCII vs Unicode)

3.3 Easy

The following problems introduce fundamental string parsing and manipulation techniques. The emphasis is on establishing and maintaining simple invariants while iterating through the array.

3.3.1 Valid Palindrome

Difficulty: Easy

Tags: Strings, Two Pointers

ID: LC-125

Problem. Given a string s , determine whether it is a palindrome when considering only alphanumeric characters and ignoring case. Non-alphanumeric characters (punctuation/whitespace) are ignored. After filtering, the empty string is considered a palindrome.

Key idea. We need to compare the string from both ends; a two-pointer approach lets us scan inward while skipping non-alphanumeric characters, achieving $O(n)$ time and $O(1)$ extra space.

Author's Thinking.

<Informal reasoning, failed approaches, intuition-building, or mental models. This section is intentionally exploratory and may be omitted in promoted or exemplar versions of the book.>

Algorithm.

```

1  initialise left = 0, right = length(s) - 1
2  while left < right:
3      if s[left] is not alphanumeric:
4          left = left + 1
5          continue
6      if s[right] is not alphanumeric:
7          right = right - 1
8          continue
9      if lowercase(s[left]) != lowercase(s[right]):
10         return false
11     left = left + 1
12     right = right - 1
13 return true

```

Listing 5: Pseudocode

Correctness sketch. At the start of each iteration, all alphanumeric characters strictly outside the interval $[left, right]$ have already been matched (case-insensitively) with their corresponding partner on the other side. Skipping a non-alphanumeric character is safe because such characters are ignored by the problem definition. If $\text{lower}(s[left]) \neq \text{lower}(s[right])$, then an unmatched pair exists, so the string cannot be a valid palindrome and we return **False**. When the loop terminates ($left \geq right$), every relevant pair has been verified; a middle character (odd length after filtering) does not affect palindromicity.

Complexity. Each pointer moves inward at most n times, so time complexity is $O(n)$. The algorithm uses $O(1)$ extra space (in-place scanning; no filtered string constructed).

Python Implementation

```

1  # LC 125. Valid Palindrome
2
3  class Solution:
4      def isPalindrome(self, s: str) -> bool:
5          """
6          Determines if a given string is a valid palindrome, considering only
7          alphanumeric characters and ignoring case sensitivity.
8
9          Args:
10             s (str): The input string.
11
12          Returns:
13             bool: True if the string is a palindrome, False otherwise.
14          """
15         # Initialise two pointers
16         left = 0
17         right = len(s)-1
18
19         # Iterate until the left and right pointers meet
20         while (left < right):
21             if not s[left].isalnum():
22                 left += 1
23                 continue
24             if not s[right].isalnum():
25                 right -= 1
26                 continue
27             if s[left].lower() != s[right].lower():
28                 return False
29
30         left += 1

```

```
31     right -= 1
32
33     return True
```

Listing 6: Python Solution: LC-125 Valid Palindrome
